

Q. Given a grid of size $M \times N$.

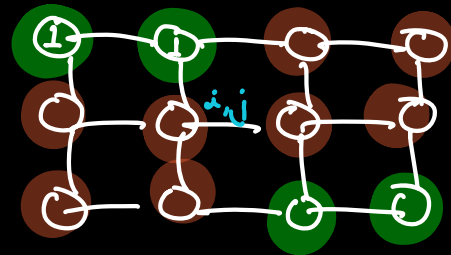
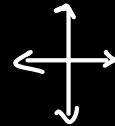
0 $\rightarrow$ water

1 $\rightarrow$ land.

Count the number of islands in the grid.

1	1	0	0	0	0	0
0	0	0	1	1	1	0
0	0	0	1	0	0	0
0	0	0	1	1	0	0
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

$\Rightarrow 3$



boolean dfs(src, dest, visited[]) {

visited[src] = true;

~~if (src == dest) return true;~~

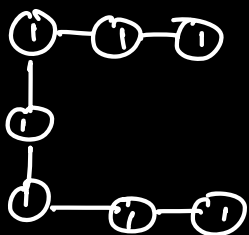
for (all nodes v in adj(src)) {

if (!visited[v] && dfs(v, dest))

return true;

return false;

}



```

void dfs (int[][] M, boolean[][] visited, int i, int j){
    if (i < 0 || j < 0 || i >= M.length || j >= M[0].length)
        return;
    if (M[i][j] == 0 || visited[i][j]) return;
    visited[i][j] = true;
    dfs (M, visited, i-1, j);
    dfs (M, visited, i+1, j);
    dfs (M, visited, i, j-1);
    dfs (M, visited, i, j+1);
}

```

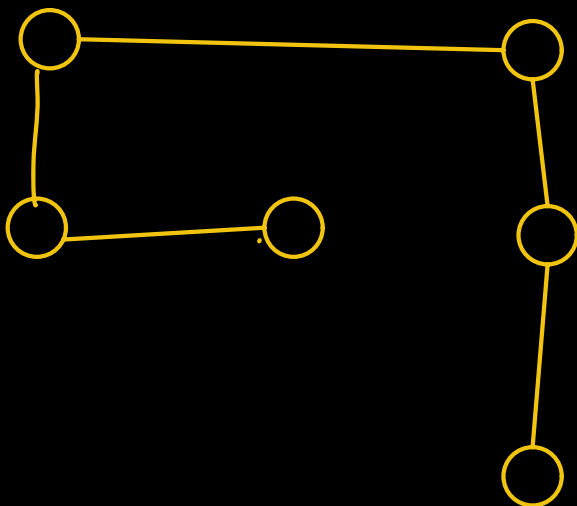
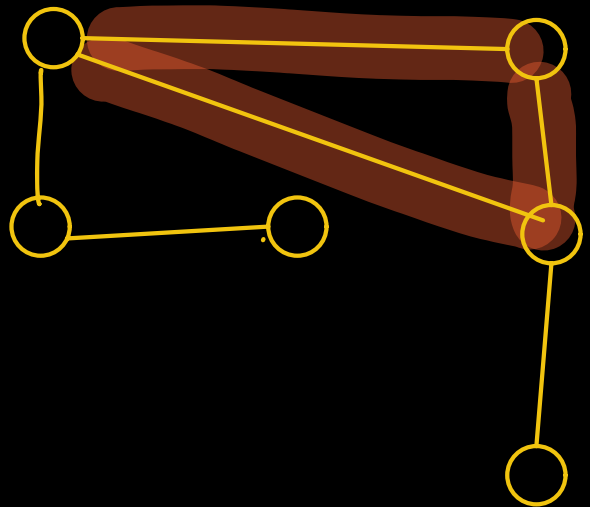
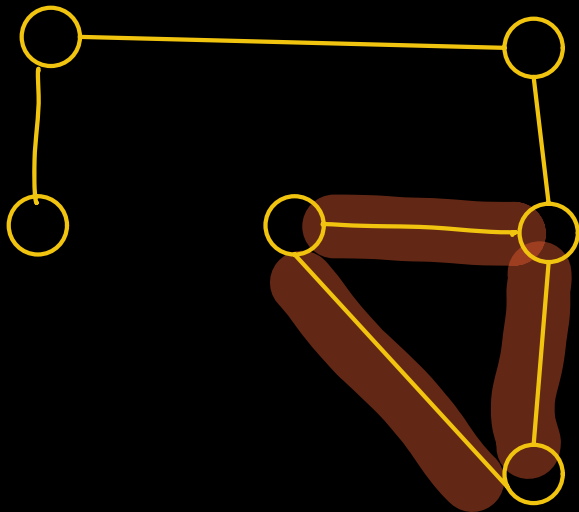
3

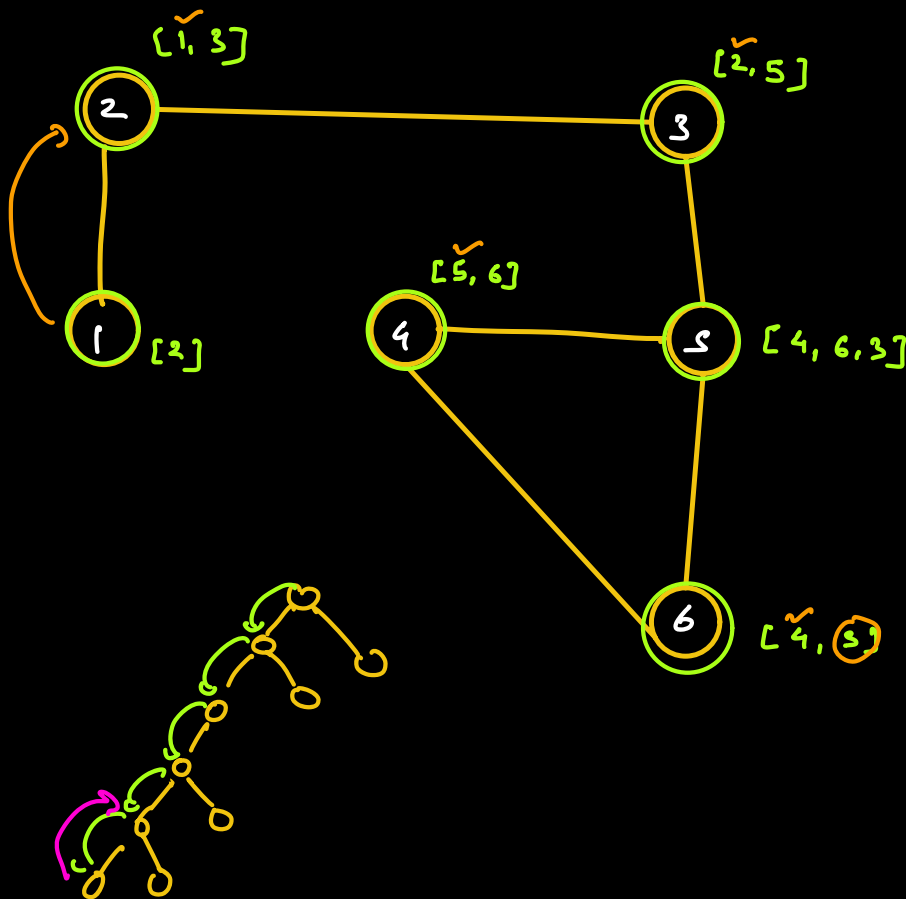
```

int noOfIsland ( M[][] ){
    ans = 0;
    visited[][] → false;
    for (i=0; i < M.length; i++){
        for (j=0; j < M[0].length; j++){
            if (M[i][j] == 1
                && !visited[i][j]){
                ans++;
                dfs (M, visited, i, j);
            }
        }
    }
    return ans;
}

```

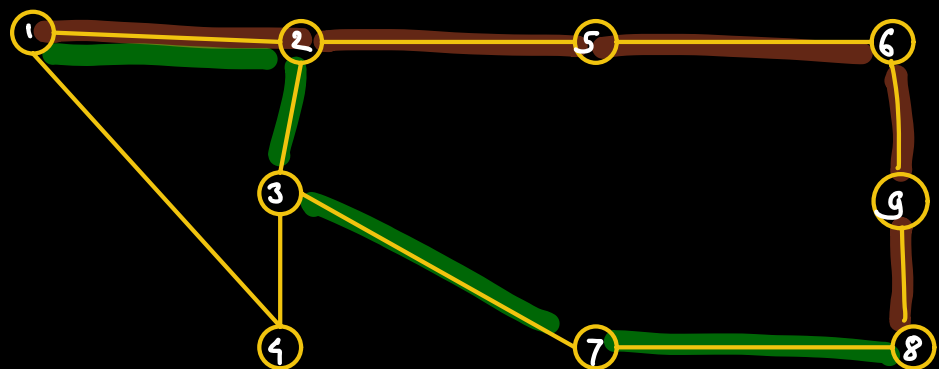
Q Given a graph. Detect if the graph has cycle.
↳ undirected, Connected.





Q Given 2 Nodes in a graph. Find the shortest distance b/w them.

S: 1
D: 8 } $\rightarrow 4$

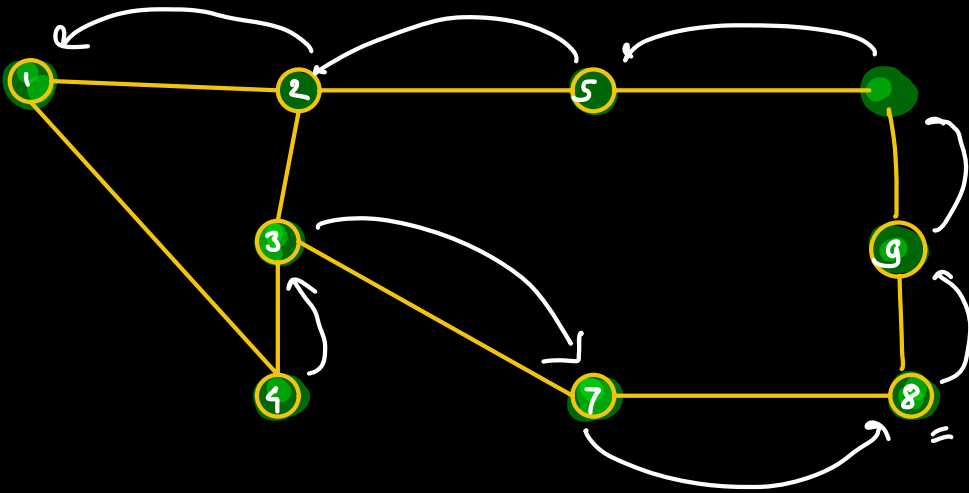


1 $\rightarrow$ 2 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7 $\rightarrow$ 8

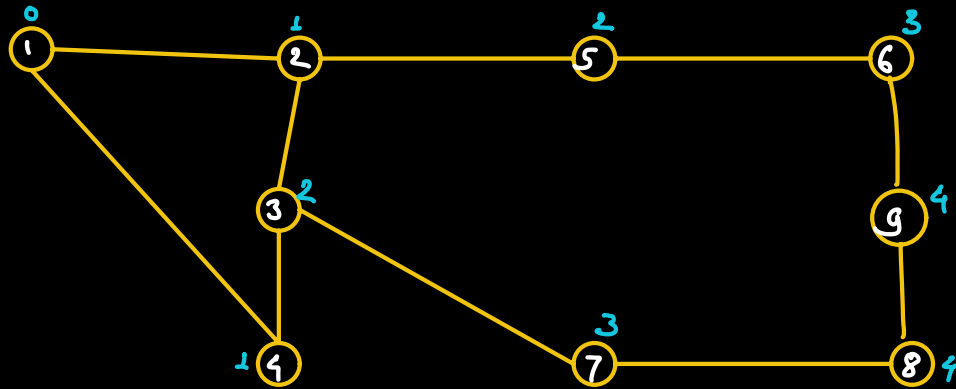
1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 7 $\rightarrow$ 8

-10

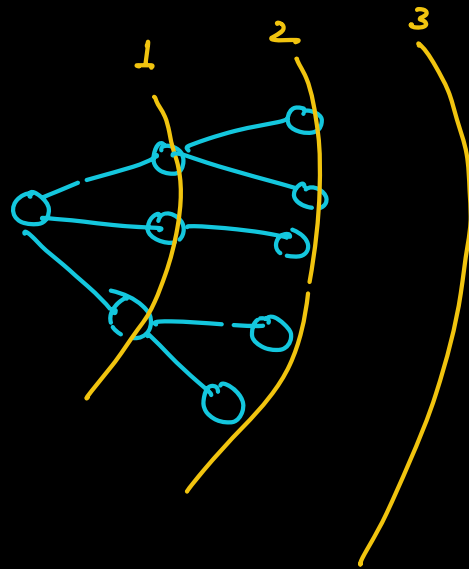
-15

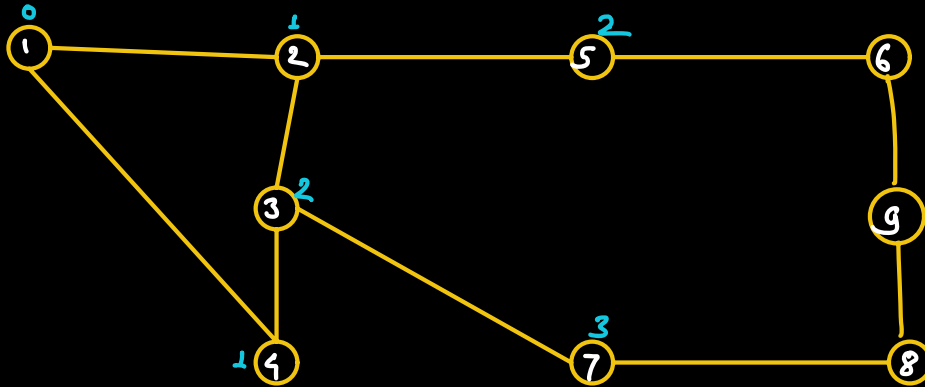
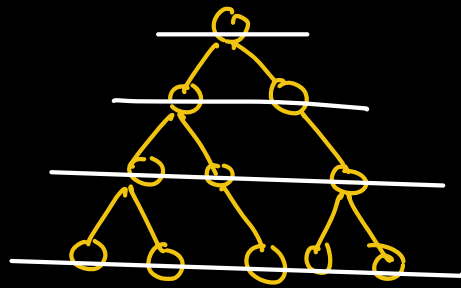


BFS



① → 2, 4 ← 1
 2 → 1, 3, 5
 3 → 2, 4, 7
 4 → 1, 3
 5 → 2, 6
 6 → 5, 9
 7 → 3, 8
 8 → 7, 9
 9 → 6, 8





visited
 $\{ 1, 2, 4 \}$

~~$\{ 1, 0 \}$~~ ~~$\{ 2, 1 \}$~~ ~~$\{ 4, 1 \}$~~ ~~$\{ 3, 2 \}$~~ $\{ 5, 2 \}$ $\{ 7, 3 \}$

Queue Q

visited[3] $\rightarrow$ true;

Q.add(5);

while (Q.size() > 0) {

node = q.poll();

list = map.get(node);

for (i = 0; i < list.size(); i++) {

v = list.get(i);

if (! visited[v]) {

Q.add(v);

visited[v] = true;

}

}

}

TC :

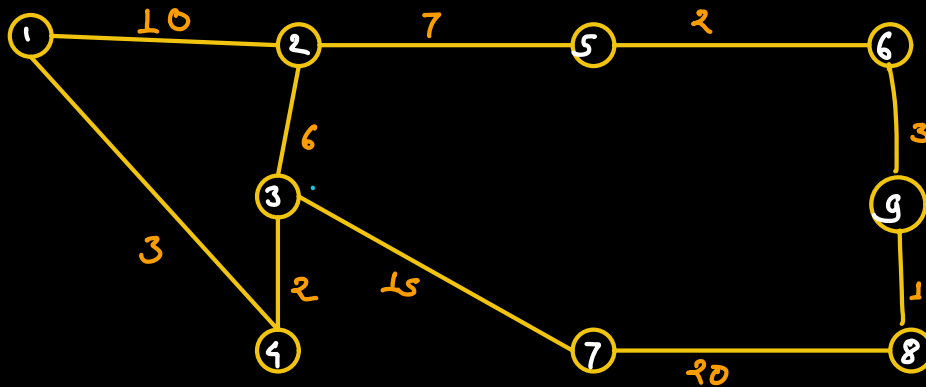
DFS $\rightarrow$?

BFS $\Rightarrow O(V+E)$

$V \rightarrow$ Nodes

$E \rightarrow$ Edges

SC:



$w > 0$